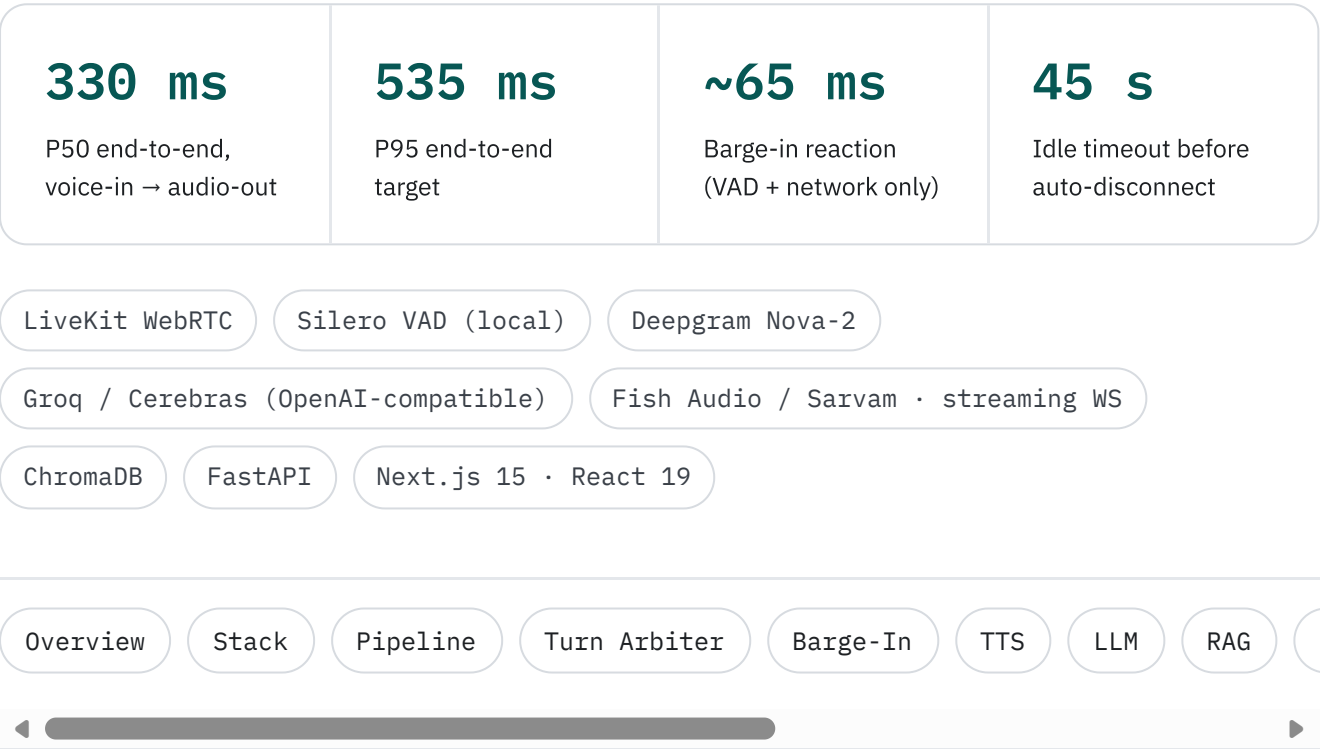


Nexus, taken apart

A real-time voice AI agent on LiveKit, FastAPI, and Next.js. The STT → LLM → TTS chain is the easy 80%; the rest of this document is the 20% that makes it feel like it's actually listening — a dual-stage semantic arbiter deciding, mid-silence, whether the user is really done talking.



Overview

Nexus is three processes talking to each other. A **FastAPI backend** (backend/main.py, port 8000) mints scoped LiveKit access tokens and ingests PDFs into a vector store. A **LiveKit agent worker** (backend/agent.py, run as python agent.py start, no exposed port — it long-polls LiveKit Cloud for job dispatches) owns the actual voice conversation: it joins the room, subscribes to the caller's audio, and runs the STT → arbiter → LLM → TTS chain. A **Next.js 15 frontend** (port 3000) is the browser client — it uploads documents, sets a system prompt, requests a token, and joins the same LiveKit room over WebRTC.

None of that is unusual for a voice agent. What's specific to this codebase is turn_arbiter.py and the way agent.py wires it in: LiveKit's AgentSession ships its own automatic endpointing (commit the turn the instant VAD reports silence), and that

own automatic endpointing (commit the turn the instant VAD reports silence), and that default is turned *off* here (turn_handling={"turn_detection": "manual"}) in favor of a custom dual-stage classifier that reads the live transcript before deciding whether silence means "done talking," "just acknowledging," or "mid-sentence pause." That decision, and the barge-in machinery hanging off it, is most of what follows.

Tech Stack

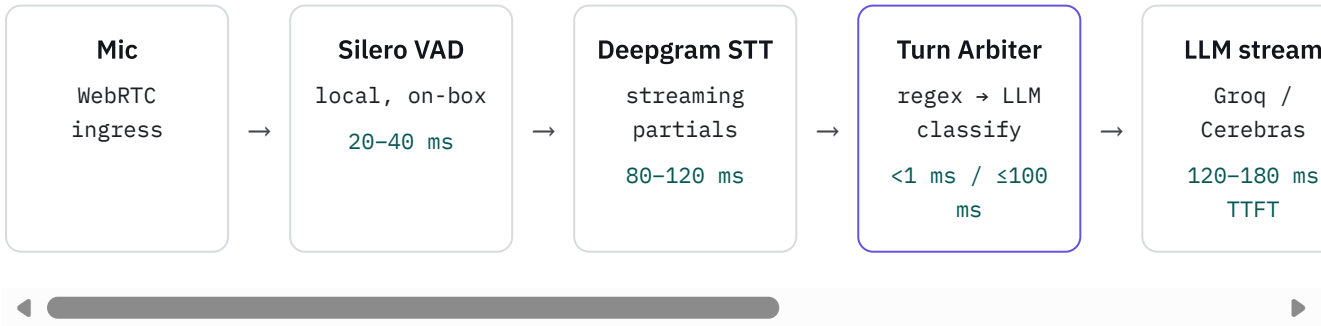
Every layer, what actually implements it in this repo, and the specific reason it was picked over the obvious alternative.

LAYER	TECHNOLOGY	WHERE
WebRTC / orchestration	LiveKit	agent.py, main.py
VAD	Silero VAD	livekit-plugins-silero
STT	Deepgram Nova-2	deepgram.STT()
LLM	Any OpenAI-compatible endpoint	llm_provider.py, llm_stream.py
TTS	Fish Audio / Sarvam	tts_streamer.py
Vector store	ChromaDB (PersistentClient)	rag.py

Backend API	FastAPI + uvicorn	main.py
Frontend	Next.js 15 · React 19 · Tailwind v4	frontend/app/page.tsx
PDF ingestion	PyPDF2 + a hand-rolled chunker	rag.py

01-08 **The Full Pipeline**

Signal flow from microphone to speaker, in the order audio actually travels. The dashed bus underneath is `InterruptController` — the barge-in kill switch every downstream stage listens to.



`InterruptController.interrupt_event` — `asyncio.Event`, fires on confirmed

`watcher` → cancels registered tasks + `session.interrupt()`

`tts_streamer` → purges queues, resets socket



STAGE	COMPONENT	P50	P95	NOTES
VAD	Silero VAD	20 ms	40 ms	Local inference; no network hop
STT	Deepgram Nova-2	80 ms	120 ms	Streaming partial transcripts
LLM	Groq (llama-3.3-70b)	120 ms	180 ms	Time-to-first-token via Groq LPU
TTS	Fish Audio / Sarvam	90 ms	150 ms	Direct WebSocket stream, first chunk
Network	LiveKit WebRTC	20 ms	45 ms	Regional relay; TURN fallback
Total	Full pipeline	330 ms	~535 ms	Barge-in cuts this to VAD + Network

WHY BARGE-IN BYPASSES THE BUDGET

Interrupting doesn't wait for a fresh LLM/TTS round trip — it fires the moment the arbiter confirms a complete turn, which only needs VAD + network to have happened. That's the ~65 ms number: everything downstream of the arbiter gets cancelled, not raced.

The Turn-Taking Arbiter

This is the "adaptive" part. `turn_arbiter.py` exists to kill two specific false positives that a naive VAD-silence-triggers-turn pipeline can't tell apart from a real turn ending.

Backchannel

"yeah", "uh-huh", "okay", "got it" — an acknowledgement said *while* the agent is talking, not a new turn.

```
→ session.clear_user_turn()
  · bot audio keeps playing
```

Mid-thought pause

Trails off on a conjunction or preposition — "...and", "because", "my..." — the user isn't done, they're thinking.

```
→ extend silence budget
  400 ms → 1200 ms,
  cancellable
```

Complete turn

The thought is finished — even a short one ("no thanks", "the blue one") counts if it's self-contained.

```
→ interrupt_ctrl.trigger()
  + session.commit_user_turn()
```

stage 1 RollingTranscriptBuffer + regex fast path

`RollingTranscriptBuffer` keeps a rolling deque of the last 4–5 seconds of Deepgram interim/final events (text, timestamp, `is_final`, confidence, whether it ends on terminal punctuation), evicting anything older than the window on every `add()`. Because Deepgram interims are cumulative re-guesses of the current utterance, the buffer's `window_text()` just returns the latest event's text.

`_fast_path_classify()` resolves the common cases deterministically, in well under a millisecond, with zero network calls:

- A 26-word backchannel list ("yeah", "yep", "uh-huh", "mm-hmm", "gotcha", "for real", "no way"...)
matched longest-first so "uh huh" doesn't get shadowed by a bare "uh".
- A dangling-connector set (and, but, so, because, or, if, when, my, your, the, to, with, of, in, is, um, uh...) — if the transcript's trailing word is in that set and there's no terminal punctuation, it's flagged as a mid-thought pause.
- A multi-word (4+) sentence ending in . ? ! is treated as obviously complete rather than spending a network round trip confirming the obvious.

Anything that doesn't confidently match one of those returns `None`, which is the caller's signal to escalate.

stage 2 Bounded structured-output LLM classification

Only genuinely ambiguous transcripts reach `classify_turn_intent()`'s LLM path — the same fast OpenAI-compatible client from `llm_stream.py` (Groq/Cerebras), asked for structured JSON:

```
// TurnClassification schema, temperature=0, max_tokens=60, response_format=json_obj  
{  
  "is_complete_turn": bool, "is_backchannel": bool, "requires_pause_extension": bool  
}
```

The system prompt ships six few-shot examples. A sample of what it's meant to resolve:

"yeah"	backchannel
"yeah, the second one"	complete turn
"so I wanted to ask about"	pause extension
"can you check my calendar for"	pause extension
"no I don't think so"	complete turn

The call is wrapped in `asyncio.wait_for(..., timeout=TURN_ARBITER_TIMEOUT_MS/1000)` — default 100 ms — and the whole thing is wrapped in a bare `except Exception`. A timeout, a provider error, or malformed JSON

never raises; it falls back to `_fallback_classify()`, the same regex heuristics as the fast path but guaranteed to return a decision. A slow or dead LLM provider literally cannot stall turn-taking.

wiring **SemanticTurnCoordinator** in `agent.py`

This required disabling `AgentSession`'s own automatic endpointing entirely — `AgentSession(turn_handling={"turn_detection": "manual"})` — since otherwise it commits or interrupts on every VAD silence or STT event before the arbiter gets a say. VAD still drives the raw events the coordinator reacts to:

- **VAD onset** (`user_state_changed` → "speaking") cancels any pending decision — the user kept talking, nothing to commit yet.
- **VAD silence** (→ "listening", already 400 ms in per Silero's `min_silence_duration`) classifies the current buffered transcript.
- A pause-extension decision sleeps the remaining 800 ms (400 → 1200 ms total), and that sleep is itself cancellable by new speech.

Every decision runs inside one cancellable `asyncio.Task`, so a new VAD onset or session teardown always wins cleanly — `CancelledError` propagates straight out of `_decide()`, cancelling the in-flight LLM call with it, with no dangling task or unclosed socket left behind.

BUG FIXED ALONG THE WAY

`@session.on("user_speech_started"/"user_speech_ended"/"agent_speech_started")` were never real `AgentSession` event names — the actual events are `user_state_changed/agent_state_changed`, each firing with an `old_state/new_state` pair. Those handlers had been silently dead code until this rewire, since wiring the arbiter required the real speaking/listening transitions to work at all.

Barge-In & the Interrupt Controller

`InterruptController` is a full-duplex `asyncio` interrupt multiplexer — one `asyncio.Event` that three independent consumers all watch.

When the arbiter's `_commit_turn()` calls `interrupt_ctrl.trigger()`, it sets `interrupt_event` — a zero-copy, sub-millisecond signal. From there:

- `_interrupt_watcher()` wakes, cancels every registered custom pipeline task (anything spawned outside `AgentSession` via `register_task()`), then calls `session.interrupt()` to flush LiveKit's own outbound audio state.
- `tts_streamer.stream_tts_worker` — watching the same event directly — purges its text and

- audio queues, then closes and reopens the provider WebSocket, since neither Fish Audio nor Sarvam expose an in-place "cancel this utterance, keep the socket" message.
- `livekit_audio_publisher.publish_audio_track` calls `AudioSource.clear_queue()` to drop audio that's already been captured into the WebRTC playout buffer but not yet played.

Registered tasks catch their own `CancelledError` internally, so the watcher never has to await them post-cancellation — it just fires-and-forgets the cancellation and moves on. This is also idempotent: triggering an interrupt while the bot isn't speaking is harmless, since every consumer's purge is a no-op on empty state.

Streaming TTS Layer

`tts_streamer.py` connects directly to a provider's WebSocket streaming API instead of a batch REST call — the whole point is starting playback on the first synthesized clause, not the finished utterance.

`clause buffering` Character-by-character, not token-by-token

`ClauseBuffer` iterates each LLM token character-by-character, since a single token can contain zero, one, or several clause boundaries (`,` `.` `?` `!` `\n`) — tokens rarely align with punctuation. Every completed clause is flushed to the provider immediately; a `TURN_END` sentinel on the text queue drains the trailing partial clause without closing the connection, while `SHUTDOWN` tears it down for good.

PROVIDER	ENDPOINT	FRAMING	AUTH
Fish Audio	<code>wss://api.fish.audio/v1/tts/live</code>	MessagePack	<code>Authorization: Bearer</code>
Sarvam	<code>wss://api.sarvam.ai/text-to-speech/ws</code>	JSON	<code>api-subscription-</code>

`lifecycle` Persistent connection, reconnect with backoff, provider fallback

One WebSocket stays open across the *whole session*, not one per turn — `_run_session()` races three futures with `asyncio.wait(..., return_when=FIRST_COMPLETED)`: the next text-queue item, the provider's audio stream, and the shared interrupt event. Whichever resolves first decides the outcome: "shutdown", "interrupted" (purge + reconnect), or "dropped" (socket died — reconnect). A reconnect uses exponential backoff (0.25s → 0.5s → 1s → 2s, 4 attempts) before falling back to `TTS_FALLBACK_PROVIDER` if the primary stays unreachable.

DESIGN CONSTRAINT

Neither provider exposes a server-side "cancel this utterance but keep the socket open" primitive — so a barge-in reset is implemented as drain-both-queues-then-reconnect, not an in-band cancel message. That's a documented protocol limitation, not an oversight.

egress livekit_audio_publisher.py

Consumes `audio_out_queue` and re-chunks the provider's bursty byte deliveries into uniform 20 ms PCM16 frames for `rtc.AudioSource`. The `AudioSource`'s own internal playout buffer (`queue_size_ms=400`) is what actually absorbs network/provider jitter without audible stuttering — the Python side just has to keep it fed with correctly-sized frames.

LLM Streaming Layer

The voice worker's LLM is a runtime config, not a hardcoded plugin — swapping Groq for Cerebras is an env var change, never a code change.

`llm_provider.py` builds the LiveKit `openai.LLM` plugin from `LLMConfig.from_env()` (`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`) so the live voice pipeline and the standalone benchmark path in `llm_stream.py` always target the same provider. `llm_stream.py`'s `stream_to_queue()` pushes each token delta onto an `asyncio.Queue` the instant it arrives on the wire — no sentence buffering — and always pushes a `STREAM_DONE` sentinel in a `finally` block, whether the stream finished normally or raised.

`latency.py`'s `StreamLatency` records two numbers per stream: **TTFT** (construction → first token observed) and **output TPS**, measured only over the generation window (first token → last token) so it isolates decode throughput from queueing/network TTFT. `bench_llm.py` exposes this as a CLI: `python bench_llm.py "What is the capital of France?"`.

NexusAgent.llm_node Forwarding into the TTS queue without breaking AgentSession

`AgentSession` normally calls its own `tts_node` once per turn — which would mean a fresh provider `WebSocket` per response, defeating the persistent-connection design above. Instead, `NexusAgent` overrides `llm_node` purely as a forwarder: it iterates `super().llm_node(...)`, pushes every content delta onto `tts_text_queue` for clause-level synthesis, and yields the original chunks through completely unchanged — so tool-calling, chat history, and turn/state tracking all keep working exactly as they would by default. `session.output.set_audio_enabled(False)` ensures `tts_node` itself is

never invoked; the custom worker owns audio output entirely.

RAG / Knowledge Base

Upload a PDF, ask about it by voice — the retrieval loop is short and entirely local.

Ingestion

`POST /upload` → `PyPDF2.PdfReader` extracts text page-by-page → `chunk_text()` slices it into 500-character windows with a 50-character overlap (pure Python, no semantic boundary detection) → `store_chunks()` writes to a single ChromaDB collection named `knowledge_base`, IDs formatted `{filename}_{index}`.

Retrieval

The agent's `search_knowledge_base @function_tool` is called by the LLM whenever the system prompt's instructions judge it relevant. Because ChromaDB's embedding call (`DefaultEmbeddingFunction` → `all-MiniLM-L6-v2`, run via Chroma's bundled ONNX runtime) is synchronous and CPU-bound, it's offloaded with `asyncio.to_thread` — running it directly on the event loop would stall audio I/O for every other coroutine sharing that loop. The top-3 (`TOP_K=3`) nearest chunks come back joined with `---` separators as the tool's return string.

Every search also publishes a `{"type": "rag_search", "query": ...}` event on LiveKit data-channel topic `rag-status`, which is what drives the "Searching: ..." pills in the frontend's chat feed in real time.

FinOps & the Idle Watchdog

One piece of this is live in the code today (the idle watchdog); the rest is a documented design (`FINOPS.md`) with a stub entry point waiting for a Redis backend.

implemented IdleWatchdog

Disconnects the WebRTC session after `NEXUS_IDLE_TIMEOUT_SECONDS` (default 45s) of no user or agent speech activity. It's built on `asyncio.wait_for()` against an `asyncio.Event` rather than a polling loop — zero CPU while idle, sub-second reaction on expiry. Every speech event (`pulse()`) re-arms the event and resets the timer; on timeout, `ctx.room.disconnect()` cascades into the endpoint's cleanup path, which is what `run_track_cleanup()` and `tear_down_every_background_task()` do.

turns `track_cost()` and tears down every background task.

DOCUMENTED, NOT WIRED

`track_cost()` is currently a logging stub. `FINOPS.md` specifies the intended Redis key schema below — it's the contract for a future implementation, not something this code currently writes.

REDIS KEY	TYPE	MEANING
<code>nexus:tokens:prompt:{session_id}</code>	INT	Prompt tokens consumed
<code>nexus:tokens:completion:{session_id}</code>	INT	Completion tokens consumed
<code>nexus:audio:secs:{session_id}</code>	FLOAT	STT audio seconds billed
<code>nexus:tts:chars:{session_id}</code>	INT	TTS characters synthesized
<code>nexus:cost:usd:{session_id}</code>	FLOAT	Running USD estimate

RESOURCE	PROVIDER	EST. RATE
STT	Deepgram	\$0.0059 / audio-min
LLM prompt	Groq	\$0.00059 / 1K tok
LLM completion	Groq	\$0.00079 / 1K tok
TTS	Deepgram	\$0.0150 / 1K chars
WebRTC relay	LiveKit Cloud	per participant-minute, plan-dependent

Frontend

A single client component (`frontend/app/page.tsx`) with two states: a setup screen, and a connected `AgentRoom`.

Setup → token → join

The setup screen collects an optional PDF (drag-drop → `POST /upload`) and an optional system prompt. Clicking "Initialize Session" calls `GET /get-token?`

`system_prompt=...`, which mints a LiveKit `AccessToken` scoped to a fresh `room-{uuid}` with `VideoGrants(room_join=True)`, embedding the system prompt and voice choice as JSON in the token's `participant.metadata` field — that's how `agent.py`'s

voice choice as soon in the token's participant metadata field — that's how `agent.py`'s `entrypoint` reads `meta.get("system_prompt")` to override its default instructions per-session, with no separate config channel.

Live hooks in AgentRoom

Built entirely on `@livekit/components-react`: `useVoiceAssistant` for agent state + audio track, `useTranscriptions/useChat` for the conversation feed, `useLocalParticipant` to toggle the mic, and `useDataChannel("rag-status", ...)` to render live "Searching: ..." pills as the backend's RAG tool fires.

AgentOrb — real audio-reactive UI

The gradient orb isn't a canned animation while the agent speaks — it reads the agent's live `MediaStreamTrack` through a `Web Audio AnalyserNode` (`fftSize=128`), averages the frequency-bin data on every animation frame, and scales the orb ($1 + \text{amplitude} \times 0.35$) in real time. Listening and thinking states use CSS breathing/rotation animations instead, since there's no output amplitude to drive during those.

Testing & Chaos Engineering

Unit tests

Four suites, all mocked — no network calls in any of them: `test_llm_stream.py`, `test_tts_streamer.py` (in-memory fake provider client), `test_turn_arbiter.py` (mocked LLM client), and `test_turn_orchestrator.py` — an end-to-end state-transition simulation against the real `SemanticTurnCoordinator` with a fake session and no LiveKit room, covering backchannel / complete-turn / pause-extension / mid-extension-interruption / LLM-escalation / clean-cancellation as separate scenarios.

Chaos playbook (CHAOS_TESTING.md)

SCENARIO	LOAD	PASS CRITERION
Baseline	1 session, 5 min	P95 E2E $\leq$ 535 ms
Ramp	1 $\rightarrow$ 10 sessions, 10 min	No drops; P95 $\leq$ 700 ms
Soak	5 sessions, 60 min	Memory stable $\pm$ 5 MB; no leaked tasks
Spike	0 $\rightarrow$ 20 in 10s	Graceful queuing; no 5xx

Fault injection covers network jitter/loss (tc netem), upstream API faults via toxiproxy (Deepgram STT timeout, Groq 429, TTS reset), and process-level kills (kill -9 on the agent worker, OOM simulation via stress). The acceptance gate checks, among other things, that InterruptController._active_tasks returns to empty between utterances and that no "Task was destroyed but it is pending!" warnings appear in logs.

Known Limitations & Tradeoffs

AREA	TRADEOFF
Vector store isolation	ChromaDB's PersistentClient means uploads survive backend restarts, but every document shares one collection — no per-session or per-user isolation.
Chunking	The character-overlap chunker is fast and dependency-free but has no semantic boundary detection — a chunk can split mid-sentence.
Clearing the KB	There's no delete endpoint; clearing the knowledge base means deleting backend/chroma_data/ directly.
CORS	allow_origins=["*"] — open for local/Docker development, must be restricted before any production deployment.
FinOps enforcement	Cost taxonomy and per-session budget caps are specified in FINOPS.md but track_cost() is a logging stub — nothing currently enforces the caps.
TTS cancel-in-place	Neither Fish Audio nor Sarvam expose a way to cancel one utterance without dropping the socket — barge-in always pays a reconnect.

Repo Map

BACKEND/ — LIVEKIT AGENT + FASTAPI TOKEN/RAG SERVER	
agent.py	Entrypoint, InterruptController, SemanticTurnCoordinator, IdleWatchdog, NexusAgent
main.py	FastAPI app — /health, /upload, /get-token
turn_arbiter.py	RollingTranscriptBuffer, regex fast path, classify_turn_intent()
tts_streamer.py	ClauseBuffer, FishAudioClient, SarvamClient,

	stream_tts_worker()
livekit_audio_publisher.py	Frame chunking + AudioSource publish loop
llm_provider.py	Builds the livekit-agents openai.LLM plugin from env
llm_stream.py	Raw AsyncOpenAI streaming primitive + LLMConfig
latency.py	StreamLatency — TTFT / TPS benchmark tracker
bench_llm.py	CLI benchmark against the configured LLM provider